EEL 5764: Computer Architecture

Lectures 4–5: Performance Fundamentals & Amdahl's Law

Sandip Ray
Warren B. Nelms Endowed Professor
Department of Electrical & Computer Engineering
University of Florida

Required Reading: H&P Sections 1.8 & 1.9, P&H Chapter 1

Quantitative Performance: Moving Beyond Intuition

THE PARADIGM SHIFT IN COMPUTER DESIGN

The Scaling Crisis: Because Dennard scaling collapsed and clock frequency can no longer grow automatically, architects cannot rely on raw physical speed to boost execution throughput.
The Microarchitectural Burden: Since physical gains must be earned structurally, design choices (e.g., adding caches, branch predictors, NPUs) are increasingly complex and expensive in area and static leakage.
No Room for Guesswork: Every additional transistor must be mathematically justified. We need a strict, empirical framework to prove that a design change delivers real, measurable user speedup.

The Architect's Ultimate Goal

"The purpose of computer architecture is to design a system that maximizes performance for a target class of workloads, subject to strict power, cost, reliability, and area constraints."
— Hennessy & Patterson

Latency vs. Throughput: Two Dimensions of Performance

1. Response Time (Latency)

The time between the start and the complete execution of a single task.

- Primary Focus: Mobile phones, client PCs, embedded control units, medical devices.
- Optimization Target: Minimize execution time (seconds per program).
- Typical Actions: Building faster pipelines, deploying low-latency caches, speeding up single-thread critical paths.

2. Throughput (Bandwidth)

The total amount of work or tasks completed per unit of time.

- Primary Focus: Cloud datacenters, web servers, high-performance computing clusters, scientific simulation engines.
- Optimization Target: Maximize instruction or transaction throughput (operations per second).
- Typical Actions: Adding more CPU cores, scaling parallel memory pipelines, leveraging hardware-assisted threading.

Defining Speedup: The Reciprocal of Execution Time

MATHEMATICAL DEFINITIONS

Defining Performance: In computer architecture, performance is defined strictly as the reciprocal of execution time. A faster machine has a smaller execution time.

Performance Equation:

Performance_X = 1 / Execution_Time_X

Defining Speedup (Relative Performance): To compare two machines X and Y, we say 'Y is n times faster than X' if:

Speedup = Performance_Y / Performance_X

Speedup = Execution_Time_X / Execution_Time_Y

If Speedup is greater than 1.0, Machine Y is faster. If less than 1.0, Y is slower (representing a performance regression).

Posing a Clear Verbal Comparison

Avoid ambiguous phrasing like 'Y is 50% faster than X' unless backed by precise mathematical ratios.

If Machine X executes a program in 10 seconds, and Machine Y executes the same program in 8 seconds:

- Speedup = Execution_Time_X / Execution_Time_Y
- Speedup = 10s / 8s = 1.25

We state: 'Machine Y is 1.25 times faster than Machine X,' or 'Machine Y delivers a 25% performance improvement over Machine X.'

Measuring Execution Time: Wall-Clock vs. CPU Time

1. Elapsed (Wall-Clock) Time

The total real-world time required to complete a given task from start to finish.

- Scope of Measure: Includes CPU execution, disk accesses, main memory stalls, operating system overhead, and input/output (I/O) delays.
- System Perspective: Highly affected by multiprogramming and background tasks sharing resource queues.
- Best For: Assessing complete end-to-end user satisfaction.

2. CPU Execution Time

The actual time the processor spends actively executing instructions of the program.

- Exclusion: Excludes I/O wait times and time spent running other background application threads.

User CPU Time: Time spent executing the actual application instructions.

System CPU Time: Time spent inside the operating system executing system calls on behalf of the program.

- Best For: Isolating processor microarchitectural design gains.

MIPS and MFLOPS: Why Synthetic Metrics Can Mislead

THE PROBLEM WITH SYNTHETIC METRICS

Millions of Instructions Per Second (MIPS): Defined as Instruction Count / (Execution Time * 10^6). Why is it a poor metric?

- Different ISAs require different amounts of work per instruction. An x86 instruction might perform a complex memory-to-memory copy, while a RISC-V instruction only performs a simple register add.
- MIPS can reward a machine for executing more simple instructions that accomplish less useful work per instruction, thereby decoupling MIPS from true performance.

MIPS is sometimes jokingly called a "Meaningless Indicator of Processor Speed."

Millions of Floating-Point Operations Per Second (MFLOPS): Only applicable for floating-point intensive codes, completely failing for integer workloads like database search or compilation.

The Standard Benchmarks: SPEC

To bypass the pitfalls of MIPS/MFLOPS, the industry uses standard, real-world workloads compiled with real toolchains.

- SPEC (Standard Performance Evaluation Corporation): Curated suites of real programs (e.g., GCC, database engines, compression, image rendering).
- SPEC Ratio: Execution time on a reference baseline machine divided by execution time on the candidate machine.
- Geometric Mean is used to summarize SPEC ratios, preventing a single extreme outlier from skewing the final result.

The CPU Performance Equation: The Iron Law of Design

THE THREE CORE VARIABLES

The Iron Law of Performance: The total CPU execution time for a program is the product of three distinct, multiplicative variables:

CPU Execution Time = Instruction Count x CPI x Clock Cycle Time

The Dimensional Validation of the Equation:

Seconds / Program = (Instructions / Program) x (Clock Cycles / Instruction) x (Seconds / Clock Cycle)

Crucial Insight: An architect cannot optimize one variable in isolation. Most microarchitectural changes represent a balance among all three variables.

Understanding the Interplay

Every design choice is a tradeoff among the three variables:

- Adding a complex instruction (ISA level): Reduces Instruction Count, but increases average CPI (takes more cycles to process) and might stretch the Clock Cycle Time.
- Deepening the pipeline (Microarchitecture): Increases operating frequency (smaller Clock Cycle Time), but increases hazards which raises the average CPI.

We will spend the entire semester manipulating these variables!

Instruction Count: The Software Blueprint

WHAT DICTATES THE INSTRUCTION COUNT?

1. Application Algorithm: Dictates the raw number of logical operations that must be executed (e.g., QuickSort requires fewer operations than BubbleSort).
2. Compiler Technology: Translates high-level code (C/C++, Rust) into assembly. Modern compilers perform loop unrolling and dead code elimination to reduce Instruction Count, while instruction scheduling optimizes instruction ordering for hardware performance.
3. Instruction Set Architecture (ISA): ISA design influences dynamic instruction count; more expressive instructions may accomplish some operations using fewer architectural instructions, although the actual count depends on the program and compiler.

Hardware Influence on IC

Does physical hardware microarchitecture affect Instruction Count?

- Generally NO: Hardware simply executes the instructions generated by the compiler. It cannot change the stream of bytes in memory.
- Exception (Hardware-Compiler Co-design): Modern processors expose specific architectural features (such as vector registers, bit manipulation extensions, or matrix units) that the compiler can target, dramatically reducing the dynamic instruction stream.

Cycles Per Instruction: The Microarchitectural Driver

WHAT DICTATES CYCLES PER INSTRUCTION?

1. Pipeline Depth: Deep pipelines allow higher frequencies but introduce hazards (data, control) that increase CPI due to stall cycles.
2. Memory Hierarchy Stalls: Cache misses force the processor to wait for main memory, adding latency cycles that directly inflate the effective CPI.
3. Branch Predictor Success: Incorrect branch predictions require flushing the pipeline and refetching, which wastes execution cycles and increases CPI.
4. Execution Width: Superscalar cores can issue multiple instructions per cycle, lowering CPI below 1.0 (introducing IPC = Instructions Per Cycle = 1 / CPI).

The Architect's Sandbox

CPI is the ultimate playground for microarchitects.
While compilers and ISAs largely define the instruction stream, the microarchitect's entire job is to minimize CPI under physical constraints.

- Ideal CPI: The baseline execution rate of the pipeline (e.g., 1.0 for a simple single-issue in-order pipeline, or 0.25 for a 4-issue out-of-order design).
- Stalled CPI: The ideal CPI plus cycles wasted waiting for hazards or memory accesses to resolve.

Clock Cycle Time: The Physical Delays

WHAT DICTATES CLOCK CYCLE TIME?

1. Semiconductor Process Technology: Defines the physical size, transition delay, and threshold properties of individual silicon transistors.
2. Critical Path Depth: The longest logic path between two sequential registers (measured in logic gate levels). High-frequency design requires keeping gate depths low (e.g., <= 10 gates per stage).
3. Wire Resistance and Capacitance: As wires became thinner, interconnect delays began dominating logic gate delays. Layout floorplans are central to keeping global wires short.

The Physical Delays

The clock period (Tc) is the inverse of the operating frequency (f = 1 / Tc).

- A 1.0 GHz clock has a period of 1.0 nanosecond.
- A 4.0 GHz clock has a period of 250 picoseconds.

Every circuit element (multiplexer, adder, wire) in the critical path must complete its transition and settle before the next clock pulse arrives. If logic fails to settle, incorrect values are latched into registers, resulting in system crashes.

Abstraction Influence: Who Controls What Variable?

Abstraction Layer	Instruction Count	CPI (Cycles / Instruction)	Clock Cycle Time (Tc)
Compiler Technology	**Highly Dominant (Optimization)**	Moderate (Inst Scheduling)	None (Logic Unchanged)
Instruction Set (ISA)	**Highly Dominant (Expressiveness)**	Moderate-High (Operations)	Moderate (Decoder Logic Depth)
Microarchitecture	None (Executes Stream)	**Highly Dominant (Hazards/Stalls)**	Moderate-High (Pipeline Depth)
Process Technology	None (RTL Unchanged)	None (RTL Unchanged)	**Highly Dominant (Transistor Delay)**

Instruction Mix: Calculating Average Cycles Per Instruction

CALCULATING THE WEIGHTED AVERAGE

Instruction Classes: Real programs execute a mix of different instruction types, each taking a different number of clock cycles to complete.
Average CPI Equation:
CPI_avg = Sum_{i=1}^{n} (CPI_i x (IC_i / Total_IC))
Where:

- CPI_i represents cycles needed for instruction class i.
- (IC_i / Total_IC) is the frequency of class i in the instruction stream.

Clock Cycles Equation:
Total Clock Cycles = Sum_{i=1}^{n} (CPI_i x IC_i)

A Concrete Example Mix

Assume a simple processor with three instruction classes:

- ALU: 50% frequency, CPI 1
- Load/store: 30% frequency, CPI 2
- Floating point: 20% frequency, CPI 5

CPI_avg = (1 * 0.50) + (2 * 0.30) + (5 * 0.20)
CPI_avg = 0.50 + 0.60 + 1.00 = 2.10 cycles/instruction
This weighted math is a recurring exam topic.

Case Study 1: Posing the Compiler Tradeoff

THE TRADE-OFF ENCOUNTERED IN REAL ENGINEERING

Your software optimization team has developed a new compiler optimization. You compile a standard benchmark program and observe the following data compared to the default compiler:

- Instruction Count: Reduced by 10% compared to the default baseline.
- Cycles Per Instruction (CPI): Increased by 5% due to the insertion of slower, more complex instructions.
- Operating Clock Frequency: Remains completely unchanged.

The Architectural Question:

Does the new compiler improve overall system performance, or does it introduce a regression? By how much does execution speed change?

Calculating Compiler Speedup

THE STEP-BY-STEP DERIVATION

Let baseline machine be X (using baseline compiler), and optimized machine be Y (using new compiler).
Define parameters for Y relative to X:

- IC_Y = 0.90 x IC_X (10% reduction)
- CPI_Y = 1.05 x CPI_X (5% increase)
- T_cY = T_cX (Frequency unchanged)

Write out the Execution Time equation ratio:

Ratio = CPU_Time_Y / CPU_Time_X
Ratio = (IC_Y x CPI_Y x T_cY) / (IC_X x CPI_X x T_cX)
Ratio = (0.90 x IC_X x 1.05 x CPI_X x T_cX) / (IC_X x CPI_X x T_cX)
Ratio = 0.90 x 1.05 = 0.945
Calculate final Relative Speedup:
Speedup = 1 / 0.945 = 1.058

Takeaways

- Execution Time decreased to 94.5% of the baseline.
- Speedup is exactly 1.058x (not just the algebraic sum -10% + 5% = -5%!).
- Net Performance Gain: The system is 5.8% faster with the new compiler.
- Architectural Moral: Always work in execution time ratios to verify performance claims. Algebraic shortcuts fail because factors are multiplicative, not additive.

Memory Stall Cycles: Unveiling the Effective CPI

INTEGRATING THE MEMORY WALL

The Memory Wall reality: Under a simplified blocking-cache model, real-world processors spend a major fraction of execution cycles stalled, waiting for main memory (DRAM) accesses to complete.
Effective CPI Formula (Ideal + Stalls):

CPI_execution = CPI_ideal + Memory_Stall_Cycles

Where Memory Stall Cycles is defined as:

Stalls = (Memory_Accesses / Instruction) x Miss_Rate x Miss_Penalty

Variables Defined:

- Memory Accesses per Instruction: Total loads and stores plus instruction fetches.
- Miss Rate: Fraction of accesses that miss the cache hierarchy.
- Miss Penalty: Cycles spent retrieving from main memory.

Key Architectural Insights

- Ideal CPI: The cycles needed if every cache access is a perfect hit (zero-latency stalls).
- Memory Stall Dominance: Because main memory access delays (miss penalties) are massive—often 100 to 200 cycles—even a tiny miss rate (e.g., 2% to 4%) can completely dominate execution time.
- The Architect's Focus: Minimizing cache miss rates and reducing miss penalties are highly leveraged paths to lower effective CPI.

Challenge: Posing the Memory Stall Penalty

THE MEMORY STALL CALCULATION SCENARIO

You are designing a high-performance single-issue in-order core operating under a simplified blocking-memory model. Your simulator provides the following data:

- Ideal CPI (Ideal Cache Hierarchy): 1.2 cycles/instruction
- Average Memory Accesses per Instruction: 1.3 loads/stores and fetches
- Cache Miss Rate: 4% (0.04) on the unified cache
- Memory Miss Penalty: 80 cycles to fetch a block from DRAM

1. What is the effective CPI of the processor once memory stalls are integrated?
2. What fraction of the processor's execution cycles is spent sitting completely idle, waiting for DRAM to return data?

Solving the Memory Stall CPI

THE STEP-BY-STEP CALCULATION

1. Identify the input variables:
- CPI_ideal = 1.2
- Accesses_per_Inst = 1.3
- Miss_Rate = 0.04
- Miss_Penalty = 80 cycles

2. Calculate dynamic Memory Stall Cycles per Instruction:

Stalls = Accesses_per_Inst x Miss_Rate x Miss_Penalty

Stalls = 1.3 x 0.04 x 80 = 4.16 cycles

3. Sum for overall stalled effective CPI:

CPI_execution = 1.2 + 4.16 = 5.36 cycles

4. Calculate the execution idle cycle fraction:

Idle_Fraction = Stall_Cycles / Effective_CPI

Idle_Fraction = 4.16 / 5.36 = 77.6%

The Mathematical Reality

- The Stalled CPI (5.36) is nearly 4.5x worse than the ideal CPI (1.2)!
- Under this simplified blocking-memory model, the idle fraction (77.6%) proves that the core is stalled, waiting for data, for more than three-quarters of its overall execution time.
- Memory is the bottleneck: This is a classic demonstration of the Memory Wall. It motivates why caching, prefetching, and multi-threading are central to computer architecture.

Design Tradeoff: Clock Period degradation vs. CPI Gains

THE PROPOSED HARDWARE MODIFICATION

Suppose you add an L1 cache to resolve memory stalls:

- CPI_baseline = 5.36 cycles
- CPI_new = 2.40 cycles (L1 cache reduces miss penalty/rate)

However, the larger cache and associative lookup logic add circuit latency, stretching the Clock Cycle Time by 10%:

- T_c_new = 1.10 x T_c_baseline

The Trade-off: CPI improves by ~55% but the clock period degrades by 10%. Is it worth it?

Write out the ratio of Execution Time:

Ratio = (IC_new x CPI_new x T_c_new) / (IC_base x CPI_base x T_c_base)

Ratio = (IC_base x 2.40 x 1.10 x T_c_base) / (IC_base x 5.36 x T_c_base)

Ratio = (2.40 x 1.10) / 5.36 = 2.64 / 5.36 = 0.492

Design Verdict & Speedup

- Speedup = Execution_Time_base / Execution_Time_new
- Speedup = 5.36 / 2.64 = 2.03x

Verdict: YES, this modification is highly beneficial! Even with a slower clock cycle time, the system is 2.03 times faster because of the massive CPI improvement.

This is an excellent example of why a slower clock speed can still yield a much faster processor.

Amdahl's Law: The Law of Diminishing Returns

FORMULATING AMDAHL'S LAW

Amdahl's Law: Formulated by Gene Amdahl in 1967. It determines the ultimate performance improvement gained by improving a single part of a system.
The Fundamental Speedup Formula:
Speedup_overall = 1 / ((1 - f) + (f / s))
Where:

- f: The fraction of the baseline execution time that can be enhanced by the modification (0.0 <= f <= 1.0).
- s: The speedup factor of that enhancement (s > 1.0).

Crucial Core Lesson: The performance improvement is strictly limited by the unenhanced portion (1 - f) of the system.

The Law's Central Moral

Make the common case fast!
If you build a highly complex hardware unit that speeds up floating-point division by 100x, but floating-point division represents only 1% of your application's execution time:

- f = 0.01
- s = 100
- Speedup_overall = 1 / ((1 - 0.01) + (0.01 / 100)) = 1.01x

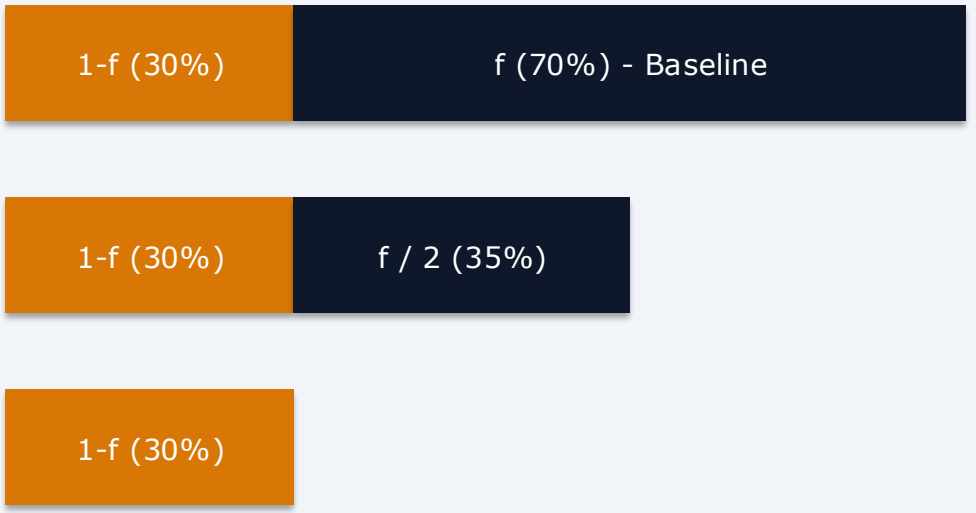
You spent massive area and power budgets for a negligible 1.0% overall performance improvement.

Visualizing Speedup Limits: The Sequential Bottleneck

VISUAL ANALYSIS OF SPEEDUP LIMITS

The Sequential Bottleneck: When a system is parallelized or accelerated, only the enhanced fraction (f) shrinks in duration.
The Unenhanced Part (1 - f) remains a fixed constant, acting as an absolute floor on the execution time of the program.
Diminishing Returns: As the local speedup factor (s) grows toward infinity, the execution time of the enhanced part shrinks to near zero, but the overall speedup halts abruptly at the sequential floor.

Visualizing the Shrinking Block



Unenhanced Block (Orange): Remains a constant size, capping overall speedup.

Enhanced Block (Dark Blue): Shrinks progressively as speedup factor (s) rises, but once it vanishes, further acceleration has zero effect.

Infinite Acceleration: Asymptotic Speedup Limit

THE ASYMPTOTIC LIMIT

Analyzing Infinite Speedup: What happens if we make the enhanced fraction execute instantly? We take the mathematical limit of Amdahl's Law as s approaches infinity:

Speedup_limit = Limit_{s -> infinity} (1 / ((1 - f) + (f / s)))

Because (f / s) approaches 0, the equation collapses to:

Speedup_limit = 1 / (1 - f)

The Sequential Bottleneck: If 20% of your program's execution time is serial (cannot be accelerated), your speedup ceiling is exactly:

Speedup_limit = 1 / 0.20 = 5.0x

Theoretical Ceiling Analysis

No matter how many parallel cores or custom accelerators you build, for a fixed workload and unenhanced fraction, additional acceleration cannot overcome this floor.

- If 50% of code is unenhanced: Max Speedup = 1 / 0.50 = 2.0x
- If 10% of code is unenhanced: Max Speedup = 1 / 0.10 = 10.0x
- If 1% of code is unenhanced: Max Speedup = 1 / 0.01 = 100.0x

This proves that parallel software optimization and compiler scheduling are just as important as hardware acceleration. We must shrink the orange block!

Multicore Scaling: Amdahl's Law in Parallel Systems

SYMMETRIC VS. ASYMMETRIC MULTICORE

Applying Amdahl's Law to Multi-Core Silicon: Suppose you have a fixed silicon area budget (measured in basic core equivalents).

- Symmetric Multi-Core: Build N identical, small, simple cores. Highly efficient for highly parallelized applications (large fraction f), but single-threaded serial bottlenecks run slowly on the simple cores.
- Asymmetric Multi-Core: Build 1 massive, fast core (consuming, e.g., 4x area) paired with N simpler cores. The single massive core accelerates the serial portion (1 - f), reducing the overall sequential bottleneck.

The Silicon Area Budget

Modern systems are heterogeneous to solve this exact trade-off.

- Modern heterogeneous architectures (such as Apple's M-series, Intel Core Ultra, or ARM big.LITTLE) integrate performance-oriented cores (with wide execution windows and high frequencies) and efficiency-oriented cores (with smaller, simpler pipelines and low leakage).
- Rather than a simple serial-vs-parallel partitioning, these designs dynamically schedule threads based on workload behavior—executing latency-critical threads on performance cores and background or parallel tasks on energy-efficient cores to maximize performance within a strict power budget.

Closing Challenge: The 3nm Upgrade Dilemma

THE ARCHITECT'S QUANTITATIVE ALLOCATION PROBLEM

You have a fixed silicon area budget in a new 3nm process node. Your design team presents two mutually exclusive options:

- Option A: Allocate area to a specialized NPU. This accelerates ML matrix execution by 50x (s = 50) for 10% of the workload's total execution time (f = 0.10).
- Option B: Implement a general-purpose microarchitectural enhancement measured to improve the complete workload by a modest 1.15x (s = 1.15, f = 1.00).

Guidelines:

1. Sketch Amdahl's Law equations for both Option A and Option B.
2. Prove mathematically which design delivers higher overall speedup.
3. Discuss how workload shifts (e.g., database vs. heavy AI inference) alter the choice.